

Lista 02 - Machine Learning IMPA

Pedro Barbosa Bahia

19 de fevereiro de 2026

Conteúdo

Exercício 1

Exercício 2

Exercício 3

Exercício 3

Este exercício utiliza o dataset MNIST para comparar diferentes algoritmos de classificação de dígitos manuscritos.

Configuração inicial e carregamento dos dados

```
1 from sklearn.datasets import fetch_openml
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.naive_bayes import BernoulliNB
4 from sklearn.discriminant_analysis import
  LinearDiscriminantAnalysis
5 from sklearn.discriminant_analysis import
  QuadraticDiscriminantAnalysis
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.neural_network import MLPClassifier
9 from sklearn.metrics import confusion_matrix
10 from sklearn import svm
11 from sklearn.model_selection import train_test_split
12 import numpy as np
13 import matplotlib.pyplot as plt
14
15 def plot_digits(
16     images,
17     n_rows=2,
18     n_cols=5,
19     fig_shape=(20, 8),
20     indexes=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
21     img_shape=(28, 28),
22     labels=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
23 ):
24     fig, axs = plt.subplots(n_rows, n_cols, figsize=fig_shape)
25     ind = np.array(indexes).reshape(n_rows, n_cols)
26     if labels:
27         plt_labels = np.array(labels).reshape(n_rows, n_cols)
28         for i in range(0, n_rows):
29             for j in range(0, n_cols):
30                 if labels:
31                     axs[i, j].set_title(plt_labels[i, j], fontsize=20)
32                     axs[i, j].imshow(
33                         (images[ind[i, j]].reshape(img_shape)), cmap="
34                         plasma"
35                     )
36                     axs[i, j].axis("off")
37
38 mnist = fetch_openml("mnist_784") # Baixar os dados
39 X, y = mnist.data.to_numpy(), mnist.target.to_numpy()
```

```

39 X = X / 255 # Colocar as features em [0, 1]
40 # Divis o em treino e teste
41 X_train, X_test, y_train, y_test = train_test_split(
42     X, y, test_size=0.2, random_state=42
43 )
44 # Plot dos dígitos
45 plot_digits(
46     X, n_rows=2, n_cols=5, indexes=[21, 24, 16, 27, 26, 35, 13, 15,
47     17, 19]
48 )

```



Figura 1: Exemplos de dígitos do dataset MNIST

(a) Comparação de modelos: acurácia e tempo de execução

```

1 from sklearn.metrics import accuracy_score
2 import time
3
4 nb = BernoulliNB(force_alpha=True) # Naive Bayes com features
   bernoulli
5 lda = LinearDiscriminantAnalysis() # LDA
6 qda = QuadraticDiscriminantAnalysis(reg_param=0.01) # QDA
7 lr = LogisticRegression(random_state=42) # Regress o Log stica
8 knn = KNeighborsClassifier(n_neighbors=6) # kNN com k = 6
9 svc = svm.SVC(gamma="scale", class_weight="balanced", C=100) # SVM

```

```

10 rf = RandomForestClassifier(
11     max_depth=30, random_state=0, n_estimators=100
12 ) # Random forest
13 nn = MLPClassifier(
14     random_state=42, hidden_layer_sizes=[100], max_iter=300
15 ) # Rede neural
16
17 models = [nb, lda, qda, lr, knn, svc, rf, nn]
18
19 yhats = []
20 accuracy = []
21 time_train = []
22 time_test = []
23 i = 0
24 for model in models:
25     start_time = time.time()
26     model.fit(X_train, y_train)
27     end_time = time.time()
28     time_train.append(end_time - start_time)
29
30     start_time = time.time()
31     yhat = model.predict(X_test)
32     end_time = time.time()
33     time_test.append(end_time - start_time)
34
35     yhats.append(yhat)
36     accuracy.append(accuracy_score(y_true=y_test, y_pred=yhat))
37     i += 1

```

Resultados de acurácia e tempo de execução:

Modelo	Acurácia	Tempo Treino (s)	Tempo Teste (s)
Naive Bayes	[ACURACIA_NB]	[TEMPO_TREINO_NB]	[TEMPO_TESTE_NB]
LDA	[ACURACIA_LDA]	[TEMPO_TREINO_LDA]	[TEMPO_TESTE_LDA]
QDA	[ACURACIA_QDA]	[TEMPO_TREINO_QDA]	[TEMPO_TESTE_QDA]
Regressão Logística	[ACURACIA_LR]	[TEMPO_TREINO_LR]	[TEMPO_TESTE_LR]
kNN	[ACURACIA_KNN]	[TEMPO_TREINO_KNN]	[TEMPO_TESTE_KNN]
SVM	[ACURACIA_SVM]	[TEMPO_TREINO_SVM]	[TEMPO_TESTE_SVM]
Random Forest	[ACURACIA_RF]	[TEMPO_TREINO_RF]	[TEMPO_TESTE_RF]
Rede Neural	[ACURACIA_NN]	[TEMPO_TREINO_NN]	[TEMPO_TESTE_NN]

Tabela 1: Comparação de acurácia e tempos de execução dos modelos

Observações:

- Para a convergência do modelo de QDA, foi necessário adicionar um parâmetro de regularização `reg_param = 0.01`.
- Os dados não foram normalizados, dada a escala semelhante das features.

(b) Análise da interpretabilidade dos modelos

Para cada um dos gráficos gerados pelos códigos abaixo, explicamos o que está sendo mostrado, como isso se relaciona com as previsões do modelo em questão

e se é possível interpretar com facilidade o que está acontecendo.

(i) Parâmetros do Naive Bayes

```
1 nb_params = {}
2 for i in range(10):
3     nb_params[i] = np.exp(nb.feature_log_prob_[i])
4
5 plot_digits(nb_params)
```



Figura 2: Probabilidades dos pixels para cada classe no modelo Naive Bayes

Cada imagem representa a probabilidade dos pixels da imagem pertencerem à respectiva classe, sendo que valores altos indicam pixels recorrentes para a classe. O produto entre este vetor e a entrada indicará a probabilidade da entrada ser pertencente à classe e será utilizada para classificação. A interpretação da imagem é simples e permite a reconstituição do dígito ao qual a classe referencia.

(ii) Parâmetros do LDA

```
1 lda_params = {}
2 for i in range(10):
3     lda_params[i] = lda.means_[i]
4
5 plot_digits(lda_params)
```



../figures/3/lda_means.png

Figura 3: Médias das classes no modelo LDA

Cada uma das imagens é a média da respectiva classe no espaço de entrada. Essa imagem seria equivalente a uma amostra média, ponto central em relação às demais amostras da classe. As previsões do modelo baseiam-se em uma métrica de similaridade da entrada em relação a cada uma dessas médias. É possível interpretar com facilidade a classe relacionada a cada imagem.

(iii) Parâmetros da Regressão Logística

```
1 log_reg_params = {}
2 for i in range(10):
3     log_reg_params[i] = lr.coef_[i]
4
5 plot_digits(log_reg_params)
```



../figures/3/logistic_regression_coefs.png

Figura 4: Coeficientes do modelo de Regressão Logística

A imagem representa o coeficiente da rede. Ao multiplicarmos pela entrada, teremos o logaritmo da probabilidade da amostra pertencer à classe sobre a probabilidade dela pertencer à classe de referência. A imagem mostrada é a resposta do modelo a uma matriz de 1's. Isso pode ser interpretado como a contribuição de cada pixel para a probabilidade final. Ainda mantemos uma interpretação válida de cada classe por meio da imagem, apesar de menos clara caso comparemos com as imagens dos modelos anteriores.

(iv) Importância das features no Random Forest

```
1 rf_params = rf.feature_importances_.reshape(28, 28)
2
3 plt.axis("off")
4 plt.imshow(rf_params, cmap="plasma")
```

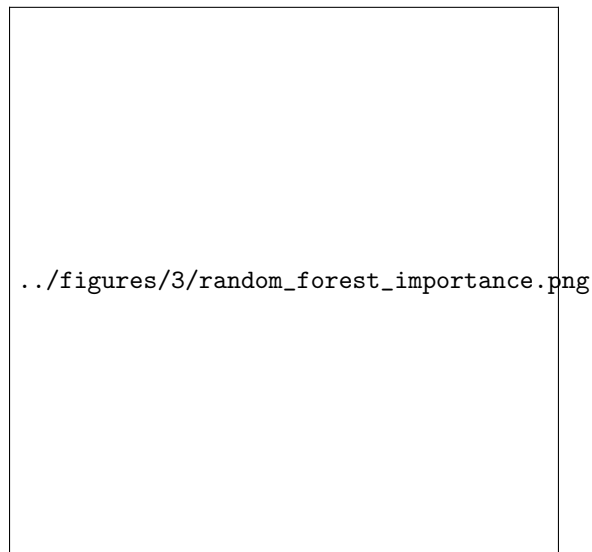



Figura 5: Importância das features no modelo Random Forest

A imagem representa a importância de cada pixel para o modelo, baseada na média do decaimento de uma métrica de impureza, no caso, o coeficiente de Gini. Valores altos indicam que aquela feature foi capaz de auxiliar na distinção entre as classes. Nesse caso, perde-se a interpretação individual das classes, entretanto ainda está clara qual a região mais importante para a classificação.

(v) Parâmetros da Rede Neural

(v.a) Pesos da camada de entrada para a camada oculta

```
1 nn_params_1 = {}
2 for i in range(100):
3     nn_params_1[i] = nn.coefs_[0][:, i]
4
5 indexes = list(range(100))
6
7 plot_digits(
8     nn_params_1,
9     n_rows=10,
10    n_cols=10,
11    fig_shape=(100, 100),
12    indexes=indexes,
13    img_shape=(28, 28),
14    labels=None,
15 )
```



Figura 6: Pesos da camada de entrada para a camada oculta na Rede Neural

Cada imagem representa os pesos que ligam a entrada a cada um dos neurônios da camada oculta. Eles são a transformação linear da entrada antes da primeira função de ativação e podem ser interpretados como parte de uma engenharia de features. A interpretabilidade já se perdeu completamente, tanto pela alta quantidade de neurônios quanto pela dificuldade de encontrar valor semântico em cada transformação.

(v.b) Pesos da camada oculta para a camada de saída

```
1 nn_params_2 = {}
2 for i in range(10):
3     nn_params_2[i] = nn.coefs_[1][:, i]
4
5 plot_digits(
6     nn_params_2, n_rows=2, n_cols=5, fig_shape=(20, 8), img_shape
7     =(10, 10)
8 )
```



Figura 7: Pesos da camada oculta para a camada de saída na Rede Neural

Cada imagem representa os pesos que ligam a camada escondida da rede à camada de saída, em que uma regressão logística será realizada. Seria semelhante ao que ocorre no modelo de regressão logística do item anterior. Entretanto, como o espaço de features é a projeção da entrada na camada intermediária, a interpretação se perde, não podendo relacionar cada classe ao numeral correspondente.

(c) Análise da matriz de confusão do Naive Bayes

```
1 cm = confusion_matrix(y_true=y_test, y_pred=yhats[0])
2 mask = 1 - np.identity(cm.shape[0])
3 error_matrix = np.multiply(cm, mask)
4
5 error_perc = error_matrix.sum(axis=1) / cm.sum(axis=1)
6
7 value, count = np.unique(y_train, return_counts=True)
```



Figura 8: Matriz de confusão do modelo Naive Bayes

Análise dos erros mais comuns:

Classe	Taxa de Erro (%)
0	[ERRO_CLASSE.0]
1	[ERRO_CLASSE.1]
2	[ERRO_CLASSE.2]
3	[ERRO_CLASSE.3]
4	[ERRO_CLASSE.4]
5	[ERRO_CLASSE.5]
6	[ERRO_CLASSE.6]
7	[ERRO_CLASSE.7]
8	[ERRO_CLASSE.8]
9	[ERRO_CLASSE.9]

Tabela 2: Taxa de erro por classe no modelo Naive Bayes

A classe mais confundida foi a [**CLASSE MAIS ERRADA**]. Nota-se pela matriz de confusão que parte considerável dos erros para esta classe ocorreram pois o modelo previu [CLASSES_CONFUNDIDAS]. Provavelmente isso acontece pela semelhança dos dígitos [DIGITOS_SIMILARES] e pelo desbalancea-

mento da classe [CLASSE_MAIIS_FREQUENTE] nos dados de treino, que foi a mais frequente.

Contagem das classes nos dados de treino:

Classe	0	1	2	3	4	5	6
Contagem	[COUNT_0]	[COUNT_1]	[COUNT_2]	[COUNT_3]	[COUNT_4]	[COUNT_5]	[COUNT_6]

Tabela 3: Distribuição das classes no conjunto de treinamento